

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №8

Тема: Патерни проектування

Виконав:
студент групи ІА-34
Єресько І.С
Перевірив:
Мягкий М.Ю

Київ-2025

Короткі теоретичні відомості

Шаблон «Composite» застосовується тоді, коли потрібно представити об'єкти у вигляді деревоподібної структури, що дозволяє описувати відношення типу «частина-ціле». Основна перевага полягає в тому, що незалежно від того, чи є об'єкт простим, чи складним і містить інші елементи, він обробляється однаковою способом. Наприклад, у типових формах інтерфейсу окремі компоненти, такі як поля вводу чи зображення, можуть містити власні вкладені елементи. Коли користувач змінює розмір форми, ця операція повинна бути застосована до всієї ієрархії компонентів, і завдяки шаблону це виконується рекурсивно та узгоджено.

Composite особливо корисний у випадках, коли потрібно оперувати ієрархією об'єктів і виконувати над ними однакові операції. Він дозволяє описувати деревоподібні структури зрозуміло й компактно, а клієнтський код взаємодії з елементами через загальний інтерфейс, не замислюючись, чи містить об'єкт дочірні елементи.

Проблемою для вирішення є побудова системи керування проєктами, де кожен проєкт складається з функцій, функції включають user story, а user story своєю чергою містять задачі. Потрібно виводити загальну оціночну вартість роботи за функцією та визначати, чи всі user story мають оцінку. Для цього зручно застосувати шаблон Composite: усі об'єкти - функції, user story і задачі -реалізують спільний інтерфейс ITask. Складні елементи містять колекцію дочірніх ITask, а задачі є листовими елементами. Метод GetEstimatedPoints() у складних елементах підсумовує значення дочірніх елементів, що дозволяє формувати оцінку незалежно від глибини структури.

Такий підхід спрощує роботу форм інтерфейсу, адже вони працюють з колекцією елементів ITask і не залежать від того, що саме перед ними -функція, user story чи проста задача. Уся логіка підрахунку зосереджена всередині відповідних класів, а клієнт просто викликає метод отримання оцінки, не аналізуючи структуру.

Composite робить роботу зі складними структурами значно гнучкішою та зручнішою. Він полегшує представлення деревоподібних моделей, дає можливість безболісно додавати та видаляти елементи, але водночас

потребує ретельного проектування інтерфейсу та додаткових зусиль на впровадження.

Шаблон «Flyweight» призначений для оптимізації використання пам'яті шляхом спільного використання однакових об'єктів у різних частинах системи. Його ключовою ідеєю є розподіл стану об'єкта на внутрішній та зовнішній. Внутрішній стан є незмінним і характерним для самого об'єкта, тому він може бути спільним для багатьох екземплярів. Зовнішній стан залежить від контексту використання й передається ззовні, що дозволяє одним об'єктом представляти багато логічних елементів.

Хорошим прикладом є система відображення тексту, де, на перший погляд, здається, що кожна літера має власний об'єкт. Насправді існує лише один об'єкт для кожного символу, а зміни координат чи оформлення передаються окремо. Це дозволяє значно економити пам'ять у системах, які працюють з великою кількістю дрібних, однотипних елементів.

Flyweight найкраще застосовувати тоді, коли програма містить велику кількість об'єктів з однаковим внутрішнім станом, наприклад графічні примітиви, символічні структури чи однотипні унікальні моделі. Попри зменшення споживання пам'яті, шаблон ускладнює код програм за рахунок введення додаткових класів і потреби розділяти стани. Також збільшується навантаження на процесор, який повинен постійно обчислювати або знаходити зовнішній контекст.

Шаблон «Interpreter» застосовується для побудови граматики спеціалізованої мови та створення інтерпретатора, який обчислює вирази цієї мови. Граматика описується у вигляді абстрактного синтаксичного дерева, де кожен вузол відповідає термінальному або нетермінальному символу. Інтерпретація відбувається рекурсивно: складні вирази спочатку обробляють свої дочірні елементи, а потім виконують власну операцію.

Цей шаблон особливо доречний у випадках, коли граматика невелика, а контекст використання простий. Це дозволяє легко масштабувати можливості мови та додавати нові правила, оскільки кожен вираз реалізується у вигляді окремого класу з методами інтерпретації.

Інтерпретатор корисний у завданнях, де потрібно часто змінювати правила обробки текстових даних або будувати гнучкі механізми пошуку,

наприклад у задачах аналізу рядків за шаблоном. Клієнт формує дерево виразів, передає контекст та викликає операцію розбору. Термінальні вузли виконують базові операції, тоді як нетермінальні відповідають за комбінацію підвиразів.

Перевагою шаблону є легке розширення граматики та можливість варіативно змінювати способи обчислень. Проте, якщо граматика стає великою, кількість класів зростає настільки, що підтримувати систему стає складно. У таких ситуаціях шаблон втрачає свою ефективність.

Шаблон «Visitor» дає можливість додавати нові операції над елементами, не змінюючи структуру самих елементів. Це дозволяє розділяти дані та логіку, зберігаючи класи чистими та не перевантаженими додатковими методами. Основна ідея полягає в тому, що об'єкти передають відвідувачу себе, а той виконує відповідну операцію залежно від їхнього типу.

Проблема добре ілюструється в онлайн-корзині інтернет-магазину. Різні товари - електроніка, хімія чи алкогольні напої - мають різні правила обробки, наприклад розрахунок вартості або формування замовлення. Якщо логіку додавати прямо в класи товарів, вони швидко стають громіздкими, а при появі нових правил доведеться змінювати всі класи товарів, що є не вигідним.

Використання Visitor дозволяє винести логіку розрахунку у спеціальні класи-відвідувачі. Кожен відвідувач реалізує окремий алгоритм, наприклад звичайний розрахунок вартості, розрахунок зі знижкою або варіант із сезонною акцією. Товари лише викликають відповідний метод у відвідувача, що робить систему масштабованою та легкою для розширення.

Головною перевагою є можливість додавати нові операції без зміни класів елементів. Недоліком є складність у додаванні нових типів елементів, адже кожен новий тип потрібно врахувати у всіх існуючих відвідувачах.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

Хід роботи

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна).

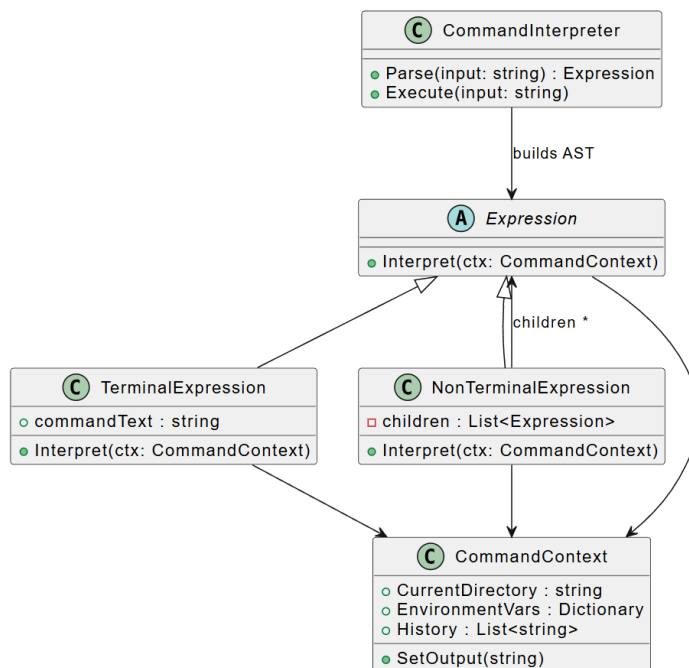


Рис 1: ClassDiagram (Interpreter)

[CommandInterpreter](#)

[Expression](#)

[TerminalExpression](#)

[NonTerminalExpression](#)

[CommandContext](#)

Клас **CommandInterpreter** є основною точкою входу: він приймає рядок введеної команди, парсить його та будує абстрактне синтаксичне дерево (AST) у вигляді об'єктів типу **Expression**.

Expression - абстрактний базовий клас, що визначає метод `Interpret(ctx: CommandContext)`. Усі вузли дерева успадковуються від нього і реалізують власну логіку виконання.

Реалізовано два типи виразів:

TerminalExpression - термінальні (лишкові) вузли, які містять конкретний текст команди і виконують прості дії (наприклад, виведення тексту, виклик вбудованої функції);

NonTerminalExpression - нетермінальні вузли, що містять список дочірніх виразів (`children`) і відповідають за складені конструкції (послідовності команд, конвеєри, умовні оператори тощо).

Під час виконання кожен вузол дерева рекурсивно викликає `Interpret()` у своїх дочірніх елементів і формує результат у спільному об'єкті

CommandContext, який зберігає: поточну директорію, змінні середовища, історію команд, буфер виводу.

Висновок:

У ході виконання роботи я ознайомився зі структурою та призначенням шаблонів проєктування «Composite», «Flyweight», «Interpreter» та «Visitor». Кожен із цих патернів вирішує окремий клас задач, пов'язаний з оптимізацією структури програми, спрощенням взаємодії між об'єктами та покращенням організації коду.

Особливу увагу було приділено шаблону «Interpreter», який я реалізував на практиці. У процесі роботи я розібрав принцип побудови граматики, структуру абстрактного синтаксичного дерева та механізм інтерпретації виразів у заданому контексті. Це дало змогу глибше зрозуміти, як будується власна мова команд та як реалізуються операції над її елементами.

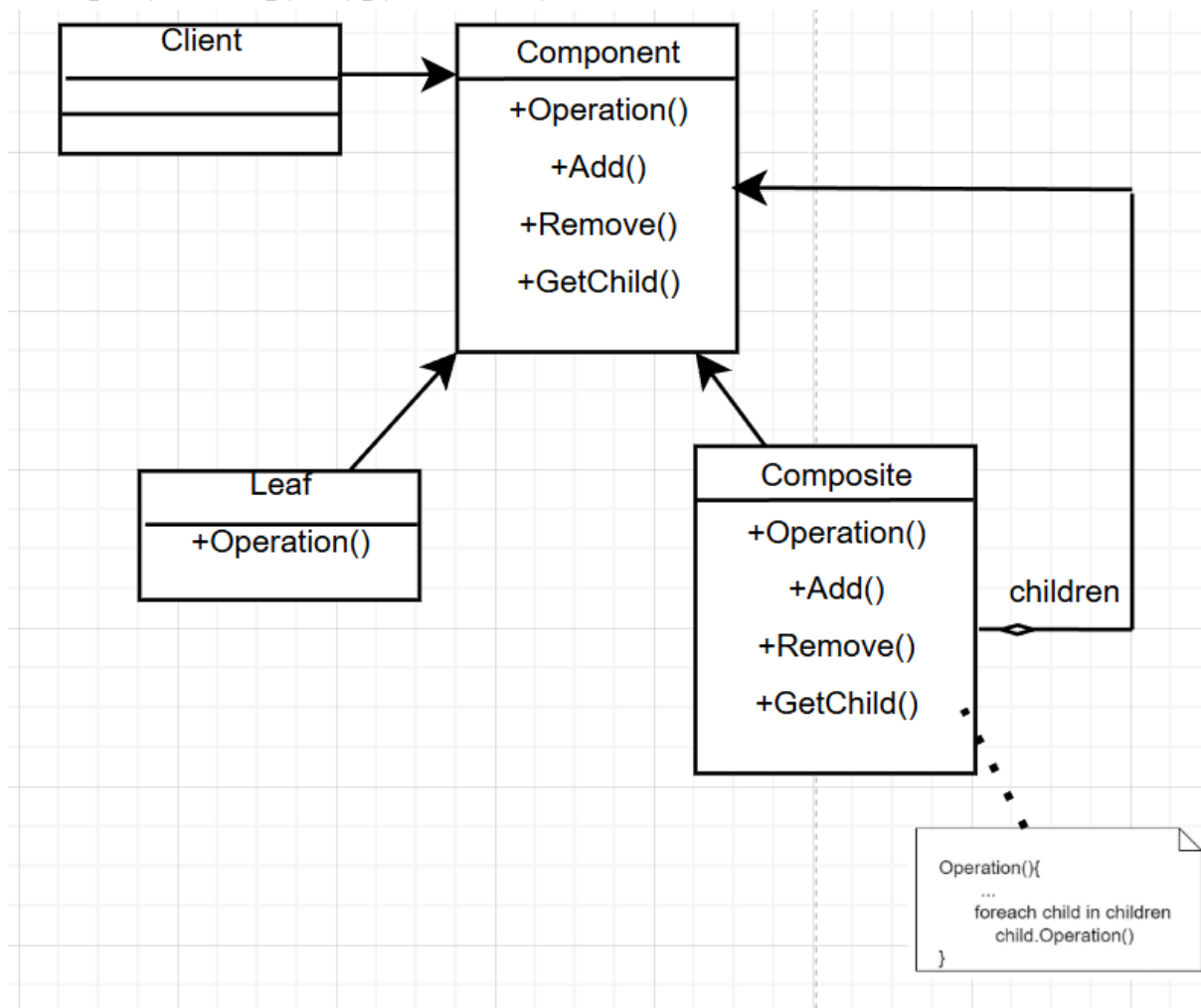
Отримані знання дозволили краще розібратися в логіці поведінкових та структурних патернів, а практична частина зі створення інтерпретатора закріпила вміння застосовувати ці шаблони в розробці програмних систем.

Контрольні запитання

1. Яке призначення шаблону «Композит»?

Шаблон «Композит» використовується для представлення складних ієрархічних структур у вигляді дерева, де кожен елемент може бути як окремим об'єктом, так і контейнером з вкладеними об'єктами. Основне призначення - уніфікувати роботу з окремими елементами і цілими групами, дозволяючи клієнту не замислюватися, має об'єкт дочірні елементи чи ні. Це значно спрощує виконання операцій над складними структурами, оскільки всі об'єкти реалізують спільний інтерфейс.

2. Нарисуйте структуру шаблону «Композит».



3. Які класи входять в шаблон «Композит», та яка між ними взаємодія?

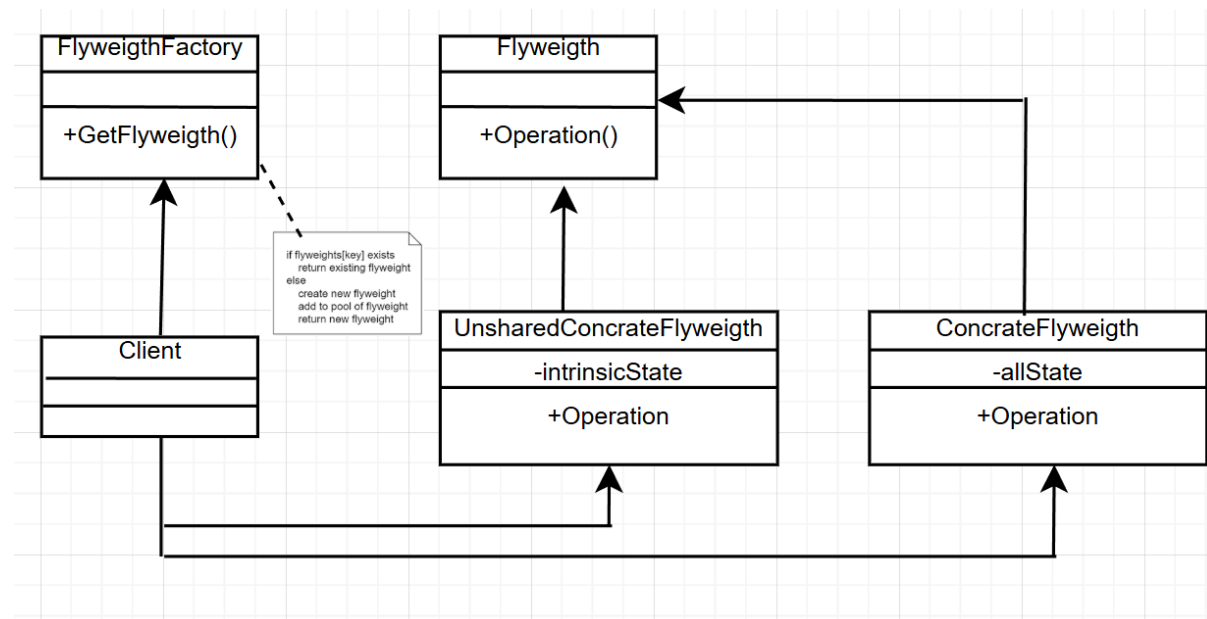
До шаблону входять три ключові класи: **Component** -спільний інтерфейс, що визначає базові операції; **Leaf** -реалізація окремого елемента, який не має вкладених об'єктів; **Composite** -елемент, який може містити багато інших **Component**. **Composite** делегує операції своїм дочірнім елементам, виконуючи їх рекурсивно. **Leaf** виконує операції без делегування. Клієнт працює лише з **Component**, не знаючи типу конкретного об'єкта.

4. Яке призначення шаблону «Легковаговик»?

Шаблон «Легковаговик» призначений для оптимізації використання пам'яті, коли в системі створюється дуже багато однакових об'єктів. Він ділить стан об'єкта на внутрішній (незмінний і спільний) та зовнішній (індивідуальний). Внутрішній стан зберігається в одному екземплярі **Flyweight**, а зовнішній передається ззовні. Завдяки цьому система може

працювати з величезною кількістю логічних об'єктів, фактично використовуючи дуже мало фізичних.

5. Нарисуйте структуру шаблону «Легковаговик».



6. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?

Основні класи: **Flyweight**, який містить внутрішній стан; **FlyweightFactory**, яка відповідає за пошук і видачу вже існуючих **Flyweight**; **Context**, що містить змінний зовнішній стан. Клієнт працює через фабрику, отримуючи спільний об'єкт **Flyweight**, а зовнішній стан передає явно. Завдяки цьому велика кількість логічних об'єктів використовує мінімальну кількість фізичних екземплярів.

7. Яке призначення шаблону «Інтерпретатор»?

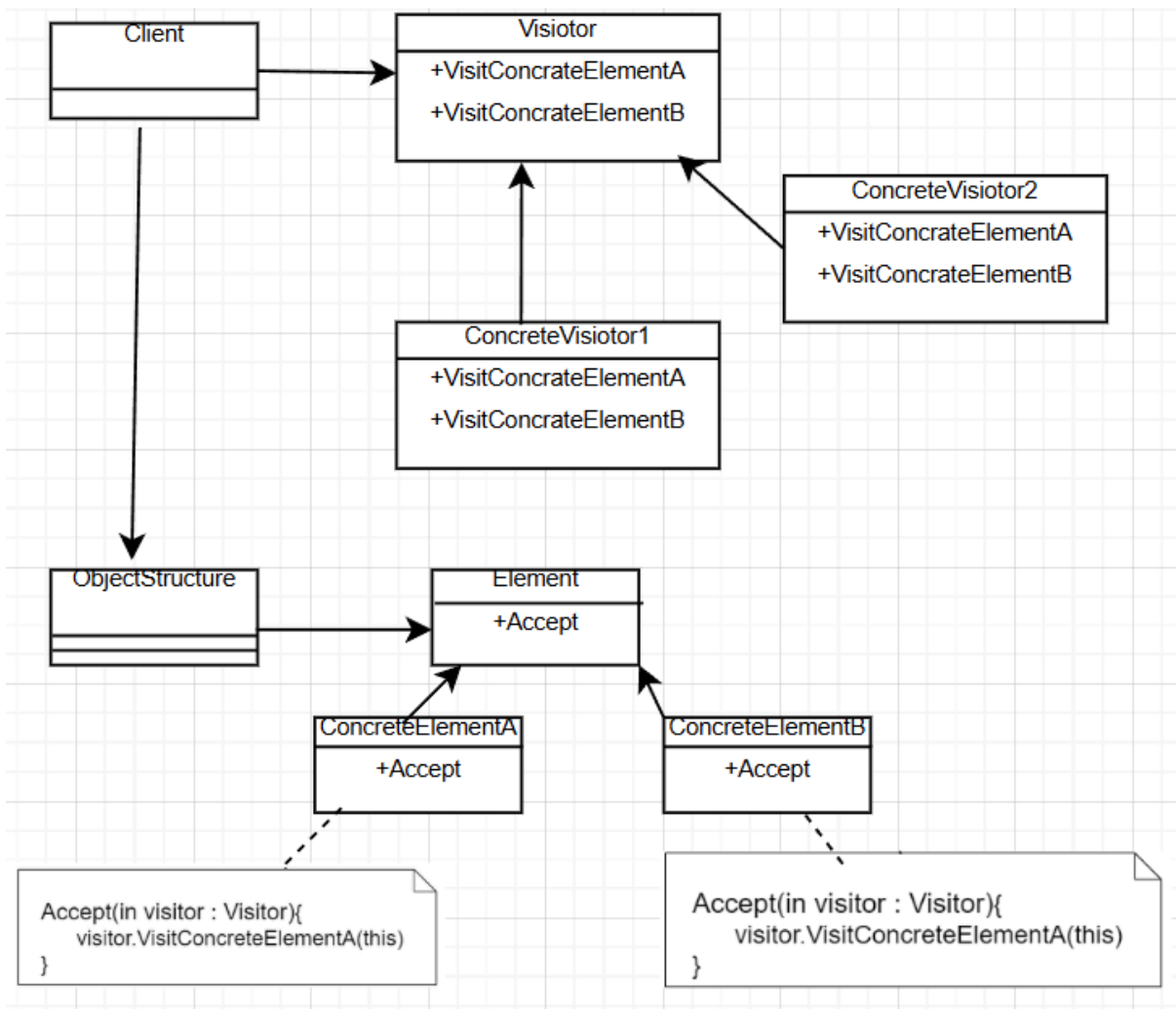
Шаблон «Інтерпретатор» використовується для визначення граматики певної мови та побудови інтерпретатора, який може обчислювати вирази цієї мови. Він створює дерево синтаксичних конструкцій, де кожен вузол відповідає термінальному або нетермінальному символу. Інтерпретація виконується рекурсивно: спочатку обчислюються дочірні вирази, а потім — операція поточного вузла. Це дозволяє просто розширювати мову та додавати нові конструкції.

8. Яке призначення шаблону «Відвідувач»?

Шаблон «Відвідувач» дозволяє визначати нові операції над об'єктами складної структури без зміни класів цих об'єктів. Він розділяє логіку та

дані: структура елементів залишається стабільною, а різні типи логік оформлюються у вигляді окремих Visitor-класів. Завдяки цьому можна додавати нові способи обробки елементів без втручання в їх код, що робить систему значно гнучкішою та легко розширюваною.

9. Нарисуйте структуру шаблону «Відвідувач».



10. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

Шаблон містить Element -базовий елемент структури, що визначає метод асерт; ConcreteElement -конкретні елементи; Visitor -інтерфейс операцій для всіх типів елементів; ConcreteVisitor -реалізації обробників. Коли елемент викликає асерт, він передає себе конкретному відвідувачу, який виконує відповідну операцію з урахуванням типу елемента. Таким чином, логіка обробки винесена окремо, а структура елементів залишається незмінною.

